

Documentação de Testes

Documentação da Estratégia de Testes

Objetivo

A estratégia de testes adotada teve como principal objetivo garantir a confiabilidade, a estabilidade e a qualidade do sistema por meio da ampliação da cobertura de testes automatizados nas camadas de backend e frontend. Para isso, foram implementados testes unitários e de integração, priorizando os módulos que concentram maior lógica de negócio e impacto funcional na aplicação.

Por decisão da equipe, o firmware foi mantido fora do escopo desta etapa, concentrando os esforços exclusivamente nas aplicações de backend e frontend.

Estratégia de Testes

Backend

No backend, foram implementados testes unitários para os métodos das camadas **Controller** e **Service**, além de testes de integração para as rotas HTTP da API de telemetria. O objetivo foi validar tanto o comportamento isolado de cada componente quanto a comunicação entre as camadas da aplicação.

Os testes contemplam diferentes cenários de execução, incluindo:

- operações realizadas com sucesso;
- tratamento de exceções e erros internos;
- validação de parâmetros de entrada;
- retornos vazios;
- exclusão de registros;
- recuperação de corridas e telemetrias.

Com a ampliação da suíte de testes, a cobertura do backend evoluiu de aproximadamente **71%** para **100%**, atendendo plenamente aos critérios estabelecidos para o projeto.

Frontend

Inicialmente, o frontend possuía apenas testes End-to-End (E2E) desenvolvidos com **Playwright**. Durante esta atividade, foi configurada uma infraestrutura dedicada à execução de testes unitários utilizando **Vitest** em conjunto com a **React Testing Library**.

Foram desenvolvidos testes para funções utilitárias, componentes React, gerenciamento de estado por meio da Context API, controle das corridas e exibição das informações de telemetria.

Os arquivos **MockFakeData.ts** e **FakeTips.ts** foram desconsiderados do cálculo da cobertura, uma vez que contêm apenas dados estáticos e definições de tipos, sem lógica de execução que justifique a implementação de testes automatizados.

Ao término da implementação, a cobertura global do frontend atingiu **99,62%**, superando com ampla margem o limite mínimo de **70%** definido para o projeto.

Ferramentas Utilizadas

Para a implementação e execução dos testes foram utilizadas as seguintes ferramentas:

Backend

- **Jest** – framework para execução de testes unitários;
- **Supertest** – realização de testes de integração nas rotas HTTP;
- **Prisma Mock** – simulação da camada de persistência de dados;
- **npm** – gerenciamento de dependências e execução dos scripts de teste.

Frontend

- **Vitest** – framework para testes unitários;
 - **React Testing Library** – testes de componentes React sob a perspectiva do usuário;
 - **jsdom** – simulação do ambiente do navegador durante a execução dos testes;
 - **Playwright** – manutenção dos testes End-to-End já existentes;
 - **npm** – gerenciamento das dependências e execução dos testes.
-

Fluxo de Execução dos Testes

Backend

A execução da suíte de testes do backend segue o seguinte fluxo:

1. acessar o diretório `src/backend`;
2. instalar as dependências do projeto utilizando `npm install`;
3. executar os testes com `npm test`;
4. gerar o relatório de cobertura utilizando `npm run test:coverage`;
5. analisar os resultados e verificar se todos os testes foram aprovados e se os índices mínimos de cobertura foram atingidos.

Frontend

Para o frontend, o fluxo de execução é semelhante:

1. acessar o diretório `src/frontend`;
 2. instalar as dependências com `npm install`;
 3. executar os testes unitários utilizando `npm test`;
 4. gerar o relatório de cobertura com `npm run test:coverage`;
 5. validar os resultados obtidos e analisar os percentuais de cobertura apresentados.
-

Novos Testes Implementados

Backend

Foram adicionados testes unitários e de integração para ampliar a cobertura da API de telemetria.

Controller

- `getRuns()`;
- `getRunById()`;
- `getTelemetriesByRunId()`.

Service

- `getRuns()`;
- `getRunById()`;
- `getTelemetriesByRunId()`.

Integração

- `GET /runs`;
- `GET /runs/:id`;

- `GET /runs/:id/telemetries.`

Frontend

Foram implementados testes unitários para os seguintes módulos:

- `utils.ts;`
- `run-context.tsx;`
- `control-panel.tsx;`
- `control-sizeMaze.tsx;`
- `navbar.tsx;`
- `minimap.tsx;`
- `telemetria-panel.tsx;`
- `table-runs.tsx.`

Esses testes validam funções utilitárias, componentes visuais, gerenciamento de estado, eventos de interação do usuário e regras de negócio presentes na interface da aplicação.

Resultados Obtidos

Após a implementação dos novos testes, foram obtidos os seguintes resultados:

Backend

- 70 testes executados;
- 70 testes aprovados;
- 100% de cobertura em **Statements, Branches, Functions e Lines.**

Frontend

- 71 testes executados;
- 71 testes aprovados;
- 99,62% de cobertura global do código.

Os resultados demonstram que a suíte de testes atende aos critérios de qualidade estabelecidos para o projeto, proporcionando elevada confiabilidade tanto para as funcionalidades do backend quanto do frontend.

Armazenamento da Documentação

Toda a documentação produzida durante esta atividade foi armazenada juntamente ao projeto e versionada no repositório, permitindo sua manutenção em conjunto com o código-fonte.

Da mesma forma, os testes implementados foram integrados à branch **develop**, garantindo que possam ser executados continuamente durante o processo de desenvolvimento e evolução do sistema.